ASSIGNMENT 6

Implementation of Data Integrity Constraints

Online Shopping Management System

1. Aim

To apply and verify integrity constraints that help maintain valid records and relationships in OnlineShoppingDB.

2. Constraints implemented

Constraint	Where used	Purpose
PRIMARY KEY	ID column in every table	Uniquely identifies each record and does not allow NULL.
FOREIGN KEY	Orders.CustomerID; OrderDetails.OrderID/ProductID; Payments.OrderID	Ensures referenced parent records exist.
UNIQUE	Customers.Email	Prevents duplicate customer email addresses.
NOT NULL	Required names, dates, prices, quantities and relationship IDs	Requires values for important fields.
CHECK	Price > 0; Stock >= 0; Quantity > 0; UnitPrice > 0; Amount > 0	Rejects invalid values in MySQL versions that enforce CHECK constraints.
DEFAULT	Products.Stock=0; Orders.Status='Pending'; Payments.PaymentStatus='Pending'	Provides a value if an INSERT omits the column.
AUTO_INCREMENT	Primary key ID columns	Generates IDs automatically when omitted.

3. Verification queries

```
USE OnlineShoppingDB;

SHOW CREATE TABLE Customers;
SHOW CREATE TABLE Products;
SHOW CREATE TABLE Orders;
SHOW CREATE TABLE OrderDetails;
SHOW CREATE TABLE Payments;

SELECT o.OrderID
FROM Orders o
LEFT JOIN Customers c ON c.CustomerID = o.CustomerID
WHERE c.CustomerID IS NULL;

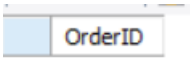
SELECT od.OrderDetailID
FROM OrderDetails od
LEFT JOIN Orders o ON o.OrderID = od.OrderID
LEFT JOIN Products p ON p.ProductID = od.ProductID
WHERE o.OrderID IS NULL OR p.ProductID IS NULL;

SELECT pay.PaymentID
FROM Payments pay
LEFT JOIN Orders o ON o.OrderID = pay.OrderID
WHERE o.OrderID IS NULL;

SELECT * FROM Products WHERE Price <= 0 OR Stock < 0;
SELECT * FROM OrderDetails WHERE Quantity <= 0 OR UnitPrice <= 0;
SELECT * FROM Payments WHERE Amount <= 0;
```

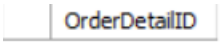
4. Outputs

Check invalid customer-order relationships



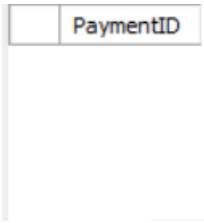
	OrderID

Check invalid order-detail relationships



	OrderDetailID

Check invalid payment relationships



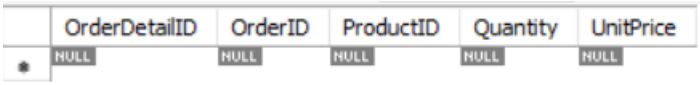
	PaymentID

Check product prices and stock



	ProductID	ProductName	Category	Price	Stock
*	NULL	NULL	NULL	NULL	NULL

Check order quantities and unit prices



	OrderDetailID	OrderID	ProductID	Quantity	UnitPrice
*	NULL	NULL	NULL	NULL	NULL

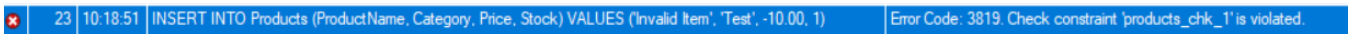
\

Check payment amounts



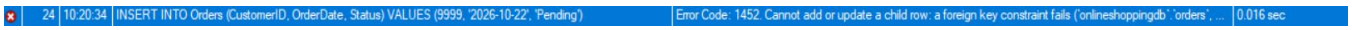
	PaymentID	OrderID	PaymentMethod	Amount	PaymentStatus
*	NULL	NULL	NULL	NULL	NULL

Test whether the UNIQUE constraint works



✖	23	10:18:51	INSERT INTO Products (ProductName, Category, Price, Stock) VALUES ('Invalid Item', 'Test', -10.00, 1)	Error Code: 3819. Check constraint 'products_chk_1' is violated.

Test Foreign Key Constraint



5. Optional negative tests

Run these statements individually only if you want to demonstrate rejected data. They are commented out so they will not accidentally execute.

```
-- Duplicate email: should be rejected by UNIQUE
-- INSERT INTO Customers (CustomerName, Email)
-- VALUES ('Test Customer', 'arun@example.com');

-- Invalid price: should be rejected by CHECK
-- INSERT INTO Products (ProductName, Category, Price, Stock)
-- VALUES ('Invalid Item', 'Test', -10.00, 1);

-- Missing parent: should be rejected by FOREIGN KEY
-- INSERT INTO Orders (CustomerID, OrderDate, Status)
-- VALUES (9999, '2026-10-22', 'Pending');
```

6. Proposed improvements

The current schema does not restrict Status, PaymentMethod, or PaymentStatus to fixed values. Add CHECK constraints or reference tables. Payments.OrderID is not UNIQUE, so multiple payment rows per order are allowed; add UNIQUE(OrderID) only if one payment per order is required.

7. Conclusion

Primary keys, foreign keys, unique values, required fields, checks, and defaults help prevent inconsistent data. Include screenshots of the actual verification results from MySQL Workbench.